

AccuSec Tenancy, Projects, Datacenters, IAM and Organizational Memory Access

Consolidated Product Requirements Document

Product: AccuSec Enterprise AI Operating System

Component: Tenancy, Project and Datacenter Management, IAM, Endpoint Identity and Entitlement Management, Organizational Memory Security

Status: Canonical Working Draft

Audience: Product Management, Enterprise Architecture, Security Architecture, IAM Engineering, Platform Engineering, AI Runtime Engineering, MCP Engineering, SRE and Compliance

1. Purpose

This PRD defines the management, tenancy, identity, authorization and Organizational Memory security model for AccuSec.

It governs:

- Tenants
- Projects
- Datacenters
- hierarchical Folders
- managed Entities
- Endpoints
- Endpoint Access Identities
- MCP Servers
- Human users
- User groups
- AI Engineer agents
- Service Principals
- Identity Providers
- Roles
- Permissions
- Access Control Policies
- Entity ownership and sharing
- Endpoint entitlement provisioning
- Provider-native IAM/RBAC synchronization
- Organizational Memory
- Organizational Context
- Audit
- Human approval

- Provider-side authorization
- Desktop, hosted single-tenant and hosted multi-tenant deployment profiles

The design must allow humans and AI Engineers to safely manage hybrid cloud and datacenter infrastructure without allowing AI reasoning to establish authority.

2. Architectural Thesis

AccuSec separates:

Probabilistic Reasoning

from:

Deterministic Enterprise Authority

An LLM may propose:

- an operation;
- a target resource;
- a workflow;
- a Skill;
- an MCP tool;
- execution parameters.

It cannot determine its own permissions.

Every protected operation must resolve:

WHO

may perform

WHICH OPERATION

on WHICH ENTITY OR ENTITY SET

inside WHICH TENANT

inside WHICH PROJECT

inside WHICH DATACENTER

within WHICH FOLDER / SCOPE

through WHICH ENDPOINT

using WHICH ENDPOINT IDENTITY

subject to WHICH POLICY

and WHICH PROVIDER AUTHORIZATION?

3. Core Security Principles

3.1 Default Deny

Absence of explicit authority means DENY.

No Project membership → DENY

No applicable Role → DENY

No applicable Permission → DENY

Wrong Datacenter → DENY

Scope mismatch → DENY

ACL mismatch → DENY

Ambiguous identity → DENY

3.2 Authentication Is Not Authorization

Successful authentication establishes identity only.

It does not automatically grant:

- Project membership;
- Datacenter access;
- Folder access;
- Entity visibility;
- MCP execution;

- Organizational Memory access.

3.3 Management Authority Is Not Infrastructure Authority

AccuSec administration and infrastructure operations are separate permission domains.

An AccuSec administrator may manage:

```
Projects
Users
Roles
Policies
Agents
Datacenters
```

without being allowed to:

```
Stop an EC2 instance
Modify an Azure VM
Expand an EBS volume
Delete a Nutanix VM
```

3.4 Human and Agent Identities Are Independent

AI Engineers are first-class security Principals.

An agent does not impersonate its human owner.

Both identities remain visible in:

- authorization;
- execution;
- audit.

3.5 Least Privilege and Delegation

A Principal may grant only permissions explicitly designated as delegable and already within that Principal's own delegation boundary.

```
Granted Permissions
⊆
Grantor Effective Delegable Permissions
```

3.6 Provider Authorization Remains Mandatory

AccuSec IAM does not replace:

- AWS IAM;
- Azure RBAC;
- GCP IAM;
- Nutanix IAM;
- VMware authorization;
- Kubernetes RBAC.

Both AccuSec and the target provider may independently deny an operation.

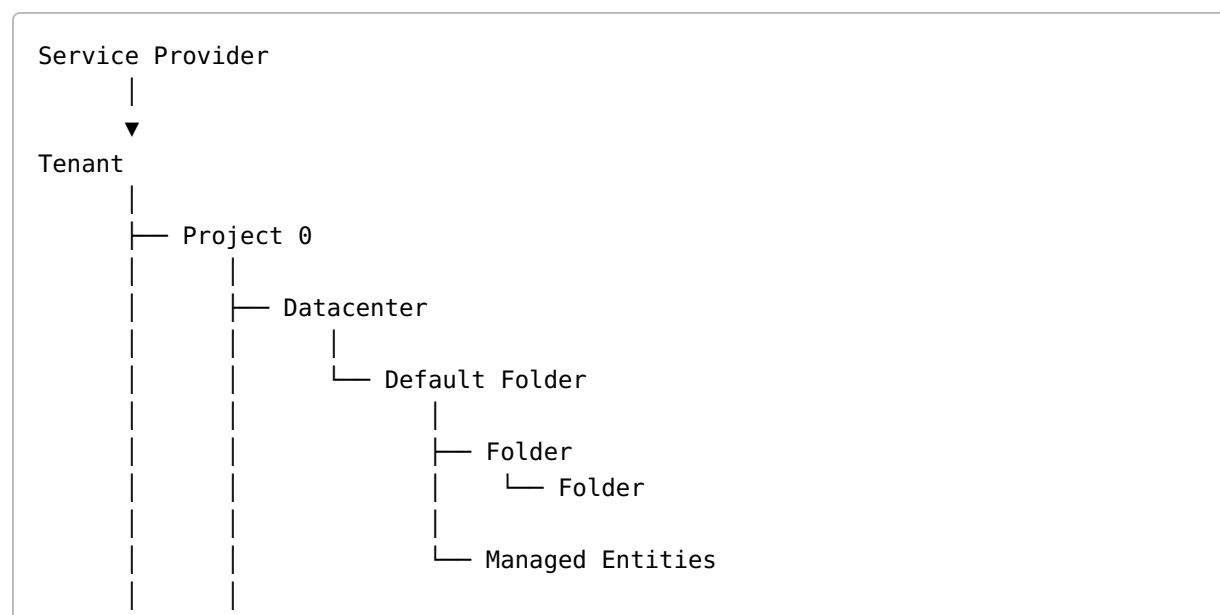
3.7 Organizational Memory Is Protected Enterprise Data

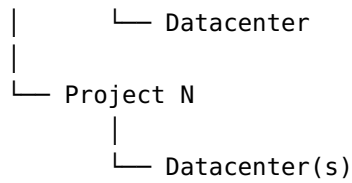
Unauthorized Organizational Memory content must never reach:

- agents;
- Skills;
- RAG retrieval;
- prompts;
- LLM context;
- generated Context Packages.

4. Canonical Management Hierarchy

The canonical hierarchy is:





The primary management and authorization scopes are:

1. Tenant
2. Project
3. Datacenter
4. Default Folder
5. Folder
6. Entity
7. Dynamic Entity Set

5. Tenant

A Tenant represents a customer or administrative security domain.

Each Tenant owns:

- Project 0
- additional Projects
- Users
- User Groups
- AI Engineers
- Identity Provider configuration
- Roles
- Permissions
- Access Control Policies
- Endpoint Connections
- Endpoint Access Identities
- Organizational Memory
- audit configuration

Each Tenant has exactly one logical Organizational Memory.

Cross-Tenant Organizational Memory access is prohibited.

6. Service Provider Administration

A Service Provider Administrator may perform AccuSec management functions according to assigned policy.

Possible capabilities include:

- creating Tenants;
- configuring Tenant identity;
- managing Project 0;
- creating Projects;
- registering Endpoints;
- configuring Datacenters;
- admitting users;
- creating Roles and Policies;
- creating agents;
- delegating administration;
- configuring audit.

An SP Admin does not automatically possess operational permissions on a customer's AWS, Azure, Nutanix or other Datacenter.

7. Project 0

Every Tenant automatically receives mandatory:

Project 0

Project 0:

- exists for the lifetime of the Tenant;
- supports users, groups and agents;
- may contain Datacenters;
- supports the same IAM semantics as other Projects.

Project 0 is not equivalent to the Tenant.

8. Projects

A Project is an administrative and operational isolation boundary within a Tenant.

A Project may contain:

- multiple Datacenters;
- multiple Endpoints through Datacenter associations;
- multiple MCP Servers;
- Human users;
- Groups;
- AI Engineers;
- Policies;
- Datacenter-specific Roles;
- Organizational Memory associations.

Example:

Project: Payments

Datacenters:

AWS Production

Azure Production

Nutanix Private Cloud

9. Datacenter

Datacenter is a first-class AccuSec object.

A Datacenter represents a provider-bound managed infrastructure domain.

The term does not imply a physical building.

Examples:

- AWS Account/environment
- Azure Subscription
- GCP Project
- Nutanix Prism Central
- VMware vCenter
- Kubernetes management domain
- private physical datacenter

Every Datacenter has:

```
datacenter_id  
tenant_id
```



```
project_id
provider_type
endpoint_reference
display_name
status
```

10. Datacenter Provider Type

Examples:

```
AWS
AZURE
GCP
NUTANIX
VMWARE
KUBERNETES
```

Provider type determines:

- provider-native hierarchy;
- entity taxonomy;
- compatible MCP Servers;
- Endpoint Identity models;
- provider operations;
- entitlement translation.

11. Datacenter Default Folder

Every Datacenter automatically contains exactly one Default Folder.

The Default Folder is:

- the Datacenter administrative root;
- the Datacenter authorization root;
- the default entity navigation location.

```
Project
├── Datacenter
│   └── Default Folder
```

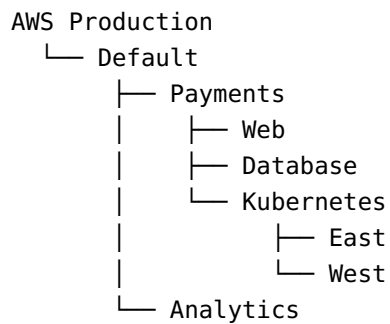
Users, groups and agents may receive permissions at:

- Project;
- Datacenter;
- Default Folder;
- descendant Folder.

12. Folder Hierarchy

Authorized administrators may create Folders at any depth beneath a Datacenter's Default Folder.

Example:



Every non-default Folder:

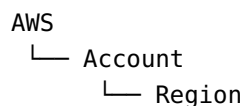
- belongs to exactly one Datacenter;
- belongs indirectly to exactly one Project;
- has one parent Folder;
- may have multiple children.

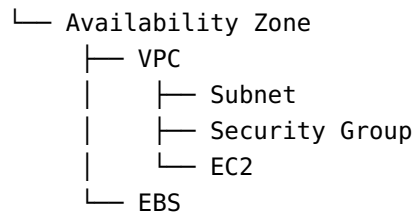
No fixed depth is imposed at the product level.

13. Native Infrastructure Hierarchy

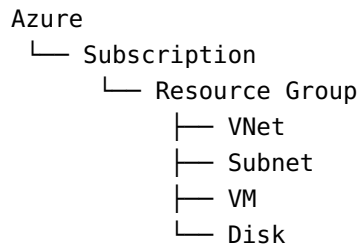
AccuSec must preserve provider-native topology.

Example AWS:

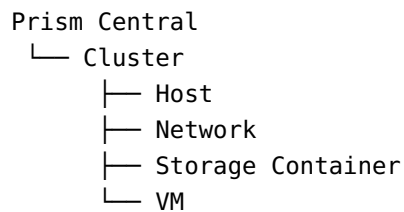




Example Azure:



Example Nutanix:



Provider-native topology and AccuSec Folder hierarchy are independent views over the same canonical Entities.

14. Provider Namespace Isolation

Entities from different Datacenters must not be ambiguously mixed.

AccuSec shall preserve explicit Datacenter context during:

- navigation;
- context retrieval;
- policy evaluation;
- MCP selection;
- Endpoint resolution;
- execution;
- audit.

A Project containing AWS and Azure must expose these as separate Datacenter namespaces.

15. Endpoint

An Endpoint represents the external managed system connection.

Examples:

- AWS environment
- Azure Subscription
- Nutanix Prism Central
- VMware vCenter
- Kubernetes API
- GCP Project

Endpoint and Datacenter are intentionally different.

Endpoint identifies the managed system.

Datacenter represents that Endpoint inside a Project's management and authorization model.

16. Endpoint Connection

Endpoint Connections are Tenant-level reusable objects.

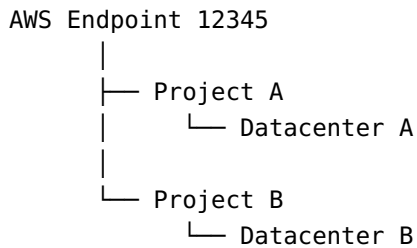
Conceptually:

```
EndpointConnection {  
    endpoint_id  
    tenant_id  
    provider_type  
    display_name  
    connection_metadata  
    authentication_type  
    status  
}
```

Secrets are referenced through a secure secrets subsystem and are not stored in ordinary configuration or Organizational Memory.

17. Endpoint Reuse Across Projects

One Endpoint may be associated with multiple Projects.



Each Project/Datacenter may have independently defined:

- users;
- agents;
- Folders;
- Policies;
- entity visibility;
- permission sets.

18. Endpoint Access Identities

A single Endpoint may support multiple Endpoint Access Identities.

Examples:

```
Infrastructure Admin Identity  
  
Coke Tenant Admin Identity  
  
Pepsi Tenant Admin Identity  
  
Read-Only Operator  
  
Agent Workload Identity  
  
Entitlement Provisioner
```

An Endpoint Access Identity represents provider-native authority, not merely credential storage.

19. Endpoint Access Identity Object

Conceptual representation:

```
EndpointAccessIdentity {  
    endpoint_identity_id  
    tenant_id  
    endpoint_id  
    provider_type  
    identity_type  
    provider_native_identity_id  
    authentication_method  
    credential_reference  
    native_role_assignments  
    resource_scope  
    status  
    last_verified_at  
}
```

20. Endpoint Identity Types

Logical classifications include:

```
INFRASTRUCTURE_ADMIN  
TENANT_ADMIN  
PROJECT_ADMIN  
OPERATOR  
READ_ONLY  
AGENT_WORKLOAD  
ENTITLEMENT_PROVISIONER  
CUSTOM
```

These are AccuSec classifications, not substitutes for provider-native Roles.

21. Service Provider Endpoint Model

A Service Provider may operate shared provider infrastructure and expose different native privilege scopes to different customers or Projects.

Example:

```
AWS Endpoint
├─ Infrastructure resources
├─ Coke resources
└─ Pepsi resources
```

Endpoint identities might include:

```
SP-Infrastructure-Role
Coke-Tenant-Admin
Pepsi-Tenant-Admin
```

These identities remain separate and cannot be substituted merely because they target the same Endpoint.

22. Infrastructure Administrator

Infrastructure Administrators may manage constructs such as:

- clusters;
- hosts;
- physical networks;
- platform storage;
- capacity;
- infrastructure lifecycle;
- platform upgrades.

They do not automatically administer Tenant virtual resources or AccuSec IAM.

23. Virtual Project Administrator

A Virtual Project Administrator manages virtual or workload infrastructure within their delegated Project/ Datacenter scope.

Possible capabilities include:

- create VMs;
- create VPCs or VNets;
- manage virtual networks;
- create virtual storage;
- manage categories/tags;

- add Project users;
- create AI Engineer agents;
- assign permitted Roles;
- create Folder hierarchy.

They do not automatically receive:

- physical host administration;
- infrastructure cluster administration;
- Service Provider IAM;
- provider organization/root access.

24. Principal Types

IAM supports:

```
HUMAN
GROUP
AI_ENGINEER
SERVICE_PRINCIPAL
```

Groups are policy subjects rather than interactive login identities.

25. Human Principals

A Human Principal contains at least:

```
principal_id
tenant_id
identity_provider_id
external_identity_id
display_name
status
```

26. AI Engineer Principals

AI Engineers are first-class non-human security Principals.

Each has:

```
agent_id
tenant_id
project_id
owner_principal_id
name
description
status
created_by
created_at
```

An AI Engineer belongs to exactly one Project.

27. Agent Project Boundary

An AI Engineer cannot operate outside its Project.

Within that Project it may receive different rights across multiple Datacenters.

```
Agent: Cloud Engineer
```

```
Project: Payments
```

```
AWS Production:
  Operator
```

```
Azure Production:
  Viewer
```

28. Agent Ownership

Every AI Engineer has one human owner.

Ownership does not automatically grant the agent all permissions held by the owner.

Agent permissions require explicit delegation.

29. Mandatory Agent Role Assignment

A newly created AI Engineer has:

Effective Permission = NONE

until an authorized human assigns Role/Policy access.

30. Agent Security Restrictions

Agents may never autonomously:

- create agents;
 - create human users;
 - create security Roles;
 - modify IAM Policies;
 - assign Roles;
 - change SSO configuration;
 - associate Identity Providers;
 - increase their privileges;
 - delegate permissions;
 - transfer entity ownership;
 - switch to a more privileged Endpoint Identity.
-

31. Identity Provider Integration

Projects may use approved enterprise identity providers.

Supported standards should include:

- OIDC
- SAML 2.0
- OAuth 2.x for delegated/API authorization

Potential providers include standards-compatible enterprise identity platforms.

32. Identity Provider Association

Authentication does not automatically create Project membership.

Required admission:

```
Approved IDP
AND
Authenticated Identity
AND
Explicit Project Admission
=
Project Member
```

33. User Groups

Policies may target:

- Human users;
- external identity groups;
- AccuSec groups where supported;
- AI Engineers.

External group membership alone does not grant Project access.

34. Roles

A Role is a named collection of permissions.

Examples:

```
AccuSec Project Administrator
Virtual Project Administrator
Cloud Viewer
AWS Operator
Storage Administrator
Network Administrator
Auditor
```

Roles alone do not grant access.

They become effective through Access Control Policies.

35. Permission Classes

AccuSec distinguishes at least four classes.

Management Permissions

```
tenant.manage  
project.create  
project.update  
datacenter.create  
folder.create  
user.add  
role.assign  
policy.manage  
agent.create  
endpoint.register  
endpoint.assign
```

Operational Permissions

```
compute.instance.start  
storage.volume.expand  
network.vpc.update
```

MCP Executable Permissions

```
mcp.aws.ec2.start_instances  
mcp.aws.ec2.modify_volume
```

Entitlement Management Permissions

```
endpoint.identity.create  
endpoint.entitlement.assign  
endpoint.entitlement.revoke  
endpoint.entitlement.reconcile
```

36. Canonical Operations

AccuSec maintains a provider-independent operation taxonomy.

Examples:

```
compute.instance.read
compute.instance.start
compute.instance.stop
compute.instance.restart

storage.volume.read
storage.volume.expand

network.vpc.read
network.vpc.update
```

37. MCP-Derived Permissions

MCP Proxy retrieves tool definitions through `GetTools`.

```
MCP Server
↓
GetTools
↓
Flatten Tool Catalog
↓
Normalize
↓
IAM Permission Catalog
```

Each executable tool maps to a fine-grained permission.

38. Tool Discovery Does Not Grant Access

Tool discovery only registers capability.

```
Tool discovered
→ Permission created
```

- Permission added to Role
- Role bound through Policy
- Principal receives scoped access

39. Multiple MCP Servers

A Project may have multiple MCP Servers.

A Datacenter may have multiple MCP Servers.

MCP Server association does not imply permission.

40. Datacenter-Bound MCP Execution

Every provider-bound MCP operation must resolve:

```
tenant_id
project_id
datacenter_id
endpoint_id
mcp_server_id
tool_id
target_entity_id
```

before execution.

41. Access Control Policy

An Access Control Policy consists of:

```
ROLE
  ↓
Permissions

SUBJECTS
  ↓
Humans / Groups / Agents
```

SCOPE



Project / Datacenter / Folder /
Explicit Entities / Scope Filter

Meaning:

The selected Subjects receive the permissions of the Role within the selected Scope.

42. Policy Scope

Scope may exist at:

- Project
- Datacenter
- Default Folder
- Folder
- Explicit Entity
- Dynamic Entity Set

43. Dynamic Scope Filters

Scope Filters resolve Entities using Organizational Memory metadata.

Example:

```
datacenter = AWS Production
AND
entity_type = aws.ec2.instance
AND
environment = production
AND
application = payments
```

44. Scope Filter Attributes

Approved attributes may include:

```
project
datacenter
folder
entity_id
entity_type
provider
endpoint
region
cluster
application
environment
owner
creator
category
label
tag
criticality
classification
```

45. Policy Line Items and Expressions

A Policy may contain multiple line items and combine them using:

```
AND
OR
UNION
NOT
```

Complex expressions use an explicit expression tree.

`NOT` is always bounded inside an already authorized Tenant/Project/Datacenter universe.

46. Delegation

Permission metadata includes:

```
delegable = true | false
```

A Principal cannot grant more than their effective delegable permission set.

This applies to:

- Human → Human
- Human → Agent

Agent → Human/Agent delegation is prohibited.

47. Project-, Datacenter- and Folder-Level Authorization

Users, groups and agents may receive scoped authorization at:

```
Project
Datacenter
Folder
Entity
Dynamic Entity Set
```

Example:

```
Alice

Project:
Payments

AWS Production:
AWS Administrator

Azure Production:
Azure Viewer
```

These permission sets remain independent.

48. Four Permission and Control Planes

The architecture contains four distinct control planes.

Plane 1 — AccuSec Management

Controls:

- Tenant
- Project
- Datacenter
- Folder
- Users
- Groups
- Agents
- Roles
- Policies
- Endpoint configuration

Plane 2 — AccuSec Operational Authorization

Determines:

Which Principal may perform which operation on which Entity in which Datacenter/Scope?

Plane 3 — Endpoint Entitlement Management

Determines:

Which provider-native identities, roles and scopes should exist to implement AccuSec operational authorization?

Plane 4 — Native Provider Enforcement

The provider ultimately evaluates its native identity, permission and resource scope.

49. Endpoint Entitlement Provisioning

When an AccuSec operational Role requires provider-side authorization, AccuSec provisions equivalent native entitlements where technically supported.

```
AccuSec Role Assignment
    ↓
Entitlement Translator
    ↓
Provider Entitlement Profile
```

50. Endpoint Entitlement Provisioning Service

A logical Endpoint Entitlement Provisioning Service provides:

- Role translation;
- identity mapping;
- resource-scope translation;
- provider Role provisioning;
- entitlement assignment;
- entitlement removal;
- synchronization;
- reconciliation;
- drift reporting.

51. AccuSec Role Versus Provider Entitlement Profile

An AccuSec Role is provider-neutral.

A Provider Entitlement Profile defines provider-specific realization.

```
AccuSec:
Virtual Compute Administrator
    |
    ├── AWS profile
    ├── Azure profile
    └── Nutanix profile
```

52. Role Assignment Lifecycle

For provider-operational access:

1. Administrator requests Role assignment

2. IAM validates grantor delegation rights
3. IAM validates Project and Datacenter
4. AccuSec records desired assignment
5. Entitlement Service selects provider profile
6. Provider identity is resolved or created
7. Provider entitlement is provisioned
8. Provider confirms
9. AccuSec verifies provider state
10. Assignment becomes ACTIVE
11. Audit event generated

53. Entitlement States

Assignments may have:

PENDING
PROVISIONING
ACTIVE
FAILED
DRIFTED
REVOKING
REVOKED

54. Revocation and Reconciliation

Removing operational permission triggers provider-side revocation where AccuSec manages the entitlement.

AccuSec periodically compares:

Desired AccuSec Entitlement

with:

Observed Provider Entitlement

Possible outcomes:

IN_SYNC
MISSING
EXTRA_PRIVILEGE
MODIFIED
IDENTITY_MISSING
SCOPE_MISMATCH
UNKNOWN

55. Provider Identity Mapping Models

Supported patterns include:

User-Mapped Identity

Alice
↓
Native Provider Principal

Shared Bounded Role

Alice / Bob
↓
AccuSec Authorization
↓
Coke-VM-Operator Role

Agent Workload Identity

AI Engineer

↓

Dedicated or bounded provider workload identity

56. Preferred Identity Strategy

Prefer:

- federation;
- short-lived credentials;
- workload identities;
- AssumeRole-style mechanisms;
- managed identities;
- service principals.

Avoid unnecessary permanent credentials.

57. Endpoint Identity Selection

Endpoint identity selection must be deterministic.

Inputs may include:

Tenant
Project
Datacenter
Principal
Role
Operation
Entity
Scope

The LLM must never select or escalate to a more privileged Endpoint Identity.

58. Entity Model

Managed Entities include infrastructure and virtual/logical resources such as:

- Clusters
- Hosts
- VMs
- VPCs
- VNets
- Subnets
- Storage
- Volumes
- Security Groups
- Categories
- Kubernetes objects
- Applications

Each has one canonical:

```
entity_id
```

Provider-native identity remains separately recorded.

59. Entity Ownership

Every Entity has one current owner.

Owner may be:

- Human Principal
- AI Engineer Principal

Track separately:

```
created_by_principal_id  
owner_principal_id
```

Ownership transfer is human-authorized.

60. Entity Sharing

Entities may be shared between permitted Folders, Datacenters or Projects.

Sharing:

- does not clone the provider resource;
- does not create a new canonical Entity ID;
- does not transfer ownership;
- does not automatically transfer source permissions.

Destination authorization is independently evaluated.

61. Organizational Memory

Each Tenant has exactly one logical Organizational Memory.

Organizational Memory represents persistent governed enterprise knowledge.

Sources may include:

- infrastructure APIs;
 - MCP Servers;
 - CMDB;
 - monitoring;
 - logs;
 - ITSM;
 - cloud managers;
 - documents;
 - PDFs;
 - spreadsheets;
 - runbooks;
 - wikis;
 - support articles;
 - incident reports;
 - tickets;
 - policies;
 - enterprise files.
-

62. Organizational Memory Metadata

Memory objects should preserve:


```
memory_object_id
tenant_id

source_type
source_system
source_id
source_reference

project_id
datacenter_id
folder associations

entity associations
entity_type
provider_type
endpoint_id

owner
creator

categories
labels
tags

classification
ACL

version
provenance
timestamps
```

63. Organizational Memory Indexing

Memory shall be logically indexable by:

```
Tenant
Project
Datacenter
Folder
Entity
Entity Type
Provider
Endpoint
```

```
Human
Group
Agent
Owner
Creator
Operation
Category
Label
Tag
Source
Document
Classification
ACL Subject
```

64. Source ACL Preservation

Enterprise documents ingested into Organizational Memory preserve applicable source access controls.

Examples:

- file ACL;
- document ACL;
- SharePoint permissions;
- Google Drive permissions;
- ticket visibility;
- wiki permissions;
- knowledge-base restrictions.

Ingestion must not broaden source access.

65. Organizational Context Service

Organizational Context is an authorized, task-specific view over Organizational Memory.

```
Organizational Memory
=
Tenant-wide persistent knowledge

Organizational Context
=
Authorized task-specific subset
```

66. Context Request

A Context request should contain:

```
tenant_id
project_id
datacenter_id
principal_id
principal_type
operation
target_entity
folder/scope
agent_id if applicable
context requirements
correlation_id
```

67. Context Authorization

Effective Context is:

```
Tenant Memory
n
Project Authorization
n
Datacenter Authorization
n
Folder Authorization
n
Entity Scope
n
Role Permissions
n
Source ACL
n
Classification Policy
=
Authorized Context
```

68. Authorization Before the LLM

Unauthorized information must be removed before model invocation.

Required:

```
graph TD;
    A[Authorize] --> B[Retrieve Permitted Data];
    B --> C[Rank];
    C --> D[Construct Context];
    D --> E[LLM];
```

RAG or vector similarity cannot bypass IAM or source ACLs.

69. IAM Policy Decision Point

Centralized IAM is the primary Policy Decision Point.

Responsibilities include:

- Principal identity
- Project membership
- Datacenter authorization
- Folder authorization
- Roles
- Permissions
- Policies
- delegation
- entity security
- agent security
- MCP permissions
- authorization decisions
- approval obligations

No LLM participates in these decisions.

70. Policy Enforcement Points

Primary enforcement points include:

```
MCP Proxy
Organizational Context Service
Management APIs
Agent Runtime / Skill Harness
Entity Management
Endpoint Entitlement Provisioning
Administrative UI/API
```

71. MCP RunTool Authorization

Before execution, MCP Proxy supplies IAM with sufficient context:

```
tenant_id
project_id
datacenter_id
folder_id

principal_id
principal_type
human_initiator_id
agent_id

endpoint_id
endpoint_identity_id

mcp_server_id
tool_id
permission_id
canonical_operation

entity_ids
scope
parameters

approval_context
correlation_id
```

72. Authorization Decision

IAM provides deterministic decisions:

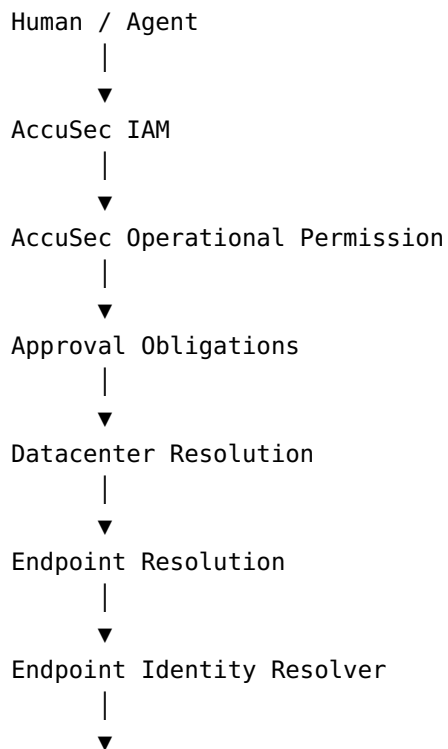
```
PERMIT
DENY
INDETERMINATE
NOT_APPLICABLE
```

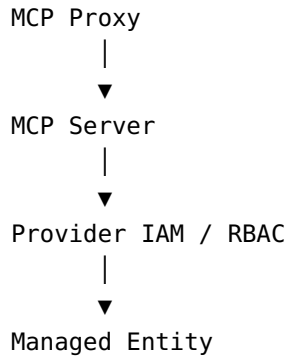
A permitted decision may carry obligations:

```
REQUIRE_APPROVAL
REQUIRE_STEP_UP_AUTH
LIMIT_PARAMETERS
REQUIRE_FRESH_CONTEXT
```

Approval remains separate from authorization.

73. Execution Authorization Sequence





74. Audit Requirements

Security-relevant events include:

- authentication;
- Project membership;
- Datacenter assignment;
- user admission/removal;
- agent creation;
- Role assignment;
- Policy changes;
- Endpoint registration;
- Endpoint identity changes;
- entitlement provisioning;
- entitlement revocation;
- entitlement drift;
- Folder changes;
- entity ownership;
- entity sharing;
- Organizational Memory access;
- Context retrieval;
- MCP invocation;
- approval;
- provider rejection.

Material events retain correlation and causation chains.

75. Day-0 / Day-1 / Day-N Model

Day 0

Service Provider / Tenant bootstrap:

- create Tenant;
- configure SSO;
- create/verify Project 0;
- register Endpoints;
- configure Endpoint provisioning identities.

Day 1

Administrative setup:

- create Projects;
- create Datacenters;
- associate Endpoints;
- create Folders;
- admit users;
- create Roles;
- define Policies;
- create agents;
- provision provider entitlements.

Day N

Operational activity:

- users and AI Engineers inspect resources;
- Context Service retrieves authorized knowledge;
- users/agents invoke MCP operations;
- approvals occur;
- provider operations execute;
- entitlements are reconciled;
- drift is reported.

76. Deployment Scale Requirement

AccuSec shall use the same logical management, IAM, Datacenter, Endpoint, Organizational Memory and execution model across all supported deployment forms.

The architecture must scale from:

Desktop Application

1 Tenant
1 Human User
Multiple Projects
Multiple Datacenters
Multiple Endpoints
Multiple MCP Servers
Multiple Folders
Multiple AI Engineers
Multiple Managed Entities

to:

Hosted / Enterprise Application

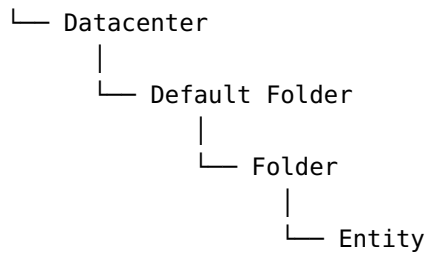
Multiple Tenants
Multiple Human Users
Multiple Groups
Multiple Projects per Tenant
Multiple Datacenters per Project
Multiple Endpoints
Multiple Endpoint Access Identities
Multiple MCP Servers
Multiple Folders
Multiple AI Engineers
Multiple Managed Entities

Desktop is a constrained deployment profile of the same architecture.

77. Canonical Hierarchy Across Deployment Profiles

All deployment modes use:

```
Tenant
|
└─ Project
    |
```



All profiles also use the same logical objects:

Endpoint Connections
Endpoint Access Identities
MCP Servers
Roles
Permissions
Policies
AI Engineers
Organizational Memory
Organizational Context
Audit

78. Desktop Deployment Profile

Desktop is optimized for one interactive operator.

Default cardinality:

Tenant:
1

Human Users:
1

Projects:
1..N

Datacenters per Project:
0..N

Folders per Datacenter:
1..N

Endpoints:

0..N

Endpoint Identities:

0..N

MCP Servers:

0..N

AI Engineers:

0..N

The Desktop application automatically creates the local Tenant and initial Human Principal.

79. Desktop Tenant and Principal

Desktop must retain both:

```
tenant_id  
principal_id
```

even though cardinality is one.

The Desktop edition shall not remove tenancy or identity from the domain model.

This preserves:

- auditability;
- ownership;
- agent delegation;
- authorization semantics;
- migration compatibility.

80. Desktop Authentication

Desktop authentication may use a simplified mechanism such as:

- local operating-system identity;
- local application identity;
- enterprise OIDC;
- enterprise SSO.

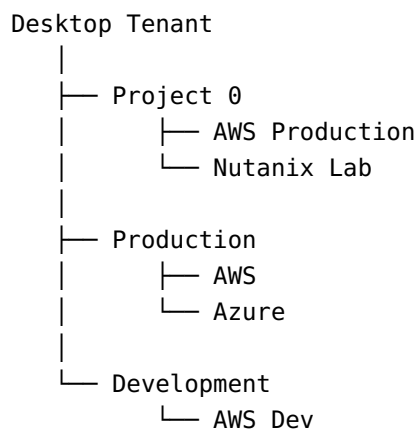
Regardless of mechanism, authentication resolves to the canonical Human Principal.

81. Desktop Projects and Datacenters

The Desktop user may create multiple Projects.

Each Project may contain multiple Datacenters.

Example:



Provider namespaces remain isolated exactly as they are in Hosted deployment.

82. Desktop Endpoints and Identities

Desktop supports multiple:

- Endpoint Connections;
- provider accounts;
- subscriptions;
- service accounts;
- Endpoint Access Identities.

For example:

```
AWS Endpoint

Identities:
  ReadOnlyRole
```

```
OperatorRole
AdministratorRole
```

The Desktop runtime must not silently choose the most privileged credential.

All identity selection rules remain deterministic.

83. Desktop IAM Simplification

Desktop may automatically grant the single Human Principal broad **AccuSec management** authority for usability.

For example:

```
Local Human

AccuSec Role:
Desktop Administrator
```

This may grant:

- Project creation;
- Datacenter creation;
- Endpoint configuration;
- Folder management;
- Role configuration;
- agent creation.

It does not automatically provide Endpoint operational authority.

A valid state is:

```
AccuSec:
Desktop Administrator

AWS:
Read Only

Azure:
No Access
```

Nutanix:
VM Operator

84. Desktop AI Engineers

Desktop supports multiple AI Engineers.

Each:

- belongs to exactly one Project;
- has the local Human Principal as owner unless otherwise supported;
- receives explicit Role/Policy assignment;
- may access one or more Datacenters in that Project;
- may have different rights per Datacenter.

85. Desktop Organizational Memory

Desktop has one Organizational Memory because it has one Tenant.

The same logical protections remain:

- Project indexing;
- Datacenter indexing;
- Folder indexing;
- Entity indexing;
- ACL metadata;
- provenance;
- Context authorization.

Some authorization checks may resolve trivially in a single-user configuration, but the security interfaces remain intact.

86. Hosted Single-Tenant Deployment

AccuSec shall support an intermediate deployment mode:

Single Tenant
Multiple Human Users
Multiple Projects

Multiple Datacenters
Multiple Folders
Multiple Agents

This mode uses the same logical architecture while enabling:

- enterprise SSO;
- groups;
- delegated administration;
- collaboration;
- centralized audit.

87. Hosted Multi-Tenant Deployment

Hosted multi-tenant deployment supports:

Tenants:

1..N

Users per Tenant:

1..N

Groups:

0..N

Projects per Tenant:

1..N

Datacenters per Project:

0..N

Folders per Datacenter:

1..N

Endpoints per Tenant:

0..N

Endpoint Access Identities:

0..N

Agents:

0..N

MCP Servers:
0..N

Every Hosted Tenant has independent:

- IAM namespace;
- Projects;
- Datacenters;
- Principals;
- Policies;
- Organizational Memory;
- Endpoint associations;
- audit scope.

88. Same IAM Engine Across Deployment Modes

Desktop and Hosted deployments use the same logical IAM request.

```
AuthorizationRequest {  
  
    tenant  
  
    project  
  
    datacenter  
  
    principal  
  
    operation  
  
    permission  
  
    entity_type  
  
    target_entities  
  
    folder_context  
  
    policy_context  
  
    endpoint_context  
  
    agent_context
```



```
    correlation_id
}
```

Desktop simply has lower cardinality for Tenant and Human Principal.

89. No Hard-Coded Single-User Assumptions

Desktop implementations shall not assume:

```
tenant_id is unnecessary
principal_id is unnecessary
authorization may be bypassed
agent owner is implicit and unrecorded
all Projects see all Entities
all Datacenters share one credential
all Endpoint identities are equivalent
```

Such shortcuts are prohibited.

90. Desktop UX May Hide Singleton Complexity

The logical model must remain enterprise-grade even when the UI is simplified.

For example, Desktop may display:

```
My AccuSec
```

rather than exposing an internal Tenant identifier.

The product principle is:

Preserve the enterprise model internally; simplify it in the product experience.

91. Logical Architecture Versus Physical Deployment

Logical components do not necessarily imply separate microservices.

Desktop may physically package:

Desktop Application

- |— IAM Module
- |— Context Module
- |— Agent Runtime
- |— MCP Proxy
- |— Endpoint Manager
- |— Entitlement Module
- |— Organizational Memory
- |— Local Persistence

within one process or a small number of processes.

Hosted deployment may physically separate them into:

- IAM Service
- Context Service
- Agent Runtime Service
- MCP Gateway
- Endpoint Service
- Entitlement Service
- Memory Service
- Audit Service

according to scale and reliability requirements.

Logical contracts remain the same.

92. Persistence Scaling

Desktop may use simplified local persistence.

Hosted deployment may use highly available distributed services.

The domain model must remain independent of physical persistence technology.

Tenant, Project, Datacenter, Principal and Entity identity must therefore remain explicit at every layer.

93. Deployment Capability Profile

AccuSec may define:

```
deployment_mode =  
    DESKTOP  
    HOSTED_SINGLE_TENANT  
    HOSTED_MULTI_TENANT
```

The profile controls:

- feature availability;
- UI complexity;
- physical service layout;
- scalability characteristics.

It does not change fundamental authorization semantics.

94. Capability Matrix

Capability	Desktop	Hosted Single Tenant	Hosted Multi-Tenant
Tenants	1	1	Many
Human Users	1	Many	Many
Projects	Many	Many	Many
Datacenters	Many	Many	Many
Folders	Many	Many	Many
Endpoints	Many	Many	Many
Endpoint Identities	Many	Many	Many
AI Engineers	Many	Many	Many
MCP Servers	Many	Many	Many
Organizational Memory	1	1	1 per Tenant

Capability	Desktop	Hosted Single Tenant	Hosted Multi-Tenant
IdP Federation	Optional	Yes	Yes
User Groups	Optional/Limited	Yes	Yes
Delegated Administration	Limited	Yes	Yes
Entitlement Reconciliation	Supported	Supported	Supported
Distributed HA	No	Optional/Yes	Yes
Multi-user Collaboration	No	Yes	Yes
Tenant Isolation	Logical singleton	Logical singleton	Mandatory

95. Desktop-to-Hosted Migration

Desktop objects must be structurally compatible with Hosted deployment.

Conceptually:

Desktop

Tenant Local

- ├ Projects
- ├ Datacenters
- ├ Folders
- ├ Entities
- ├ Endpoints
- ├ Endpoint Identities
- ├ Agents
- ├ Roles
- ├ Policies
- └ Organizational Memory

may migrate to:

Hosted Tenant

- ├ same Projects
- ├ same Datacenters
- ├ same Folders
- ├ same Entities
- ├ same Endpoint relationships
- └ same Agents

- └ same Policies
- └ migrated Organizational Memory

Additional Human Principals, Groups and enterprise IdPs may then be introduced.

Migration must not require conversion to a different security model.

96. Canonical Scale Principle

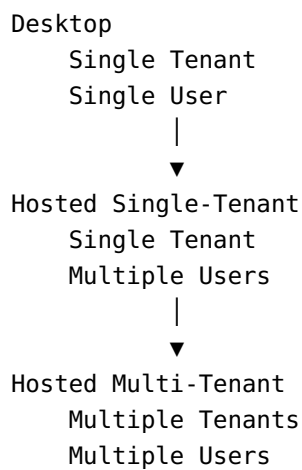
The architecture follows three foundational rules:

Single-user is cardinality 1, not a different identity model.

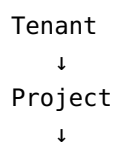
Single-Tenant is cardinality 1, not absence of tenancy.

Desktop is a deployment profile of the Enterprise AI Operating System, not a separate product architecture.

97. Product Scaling Continuum



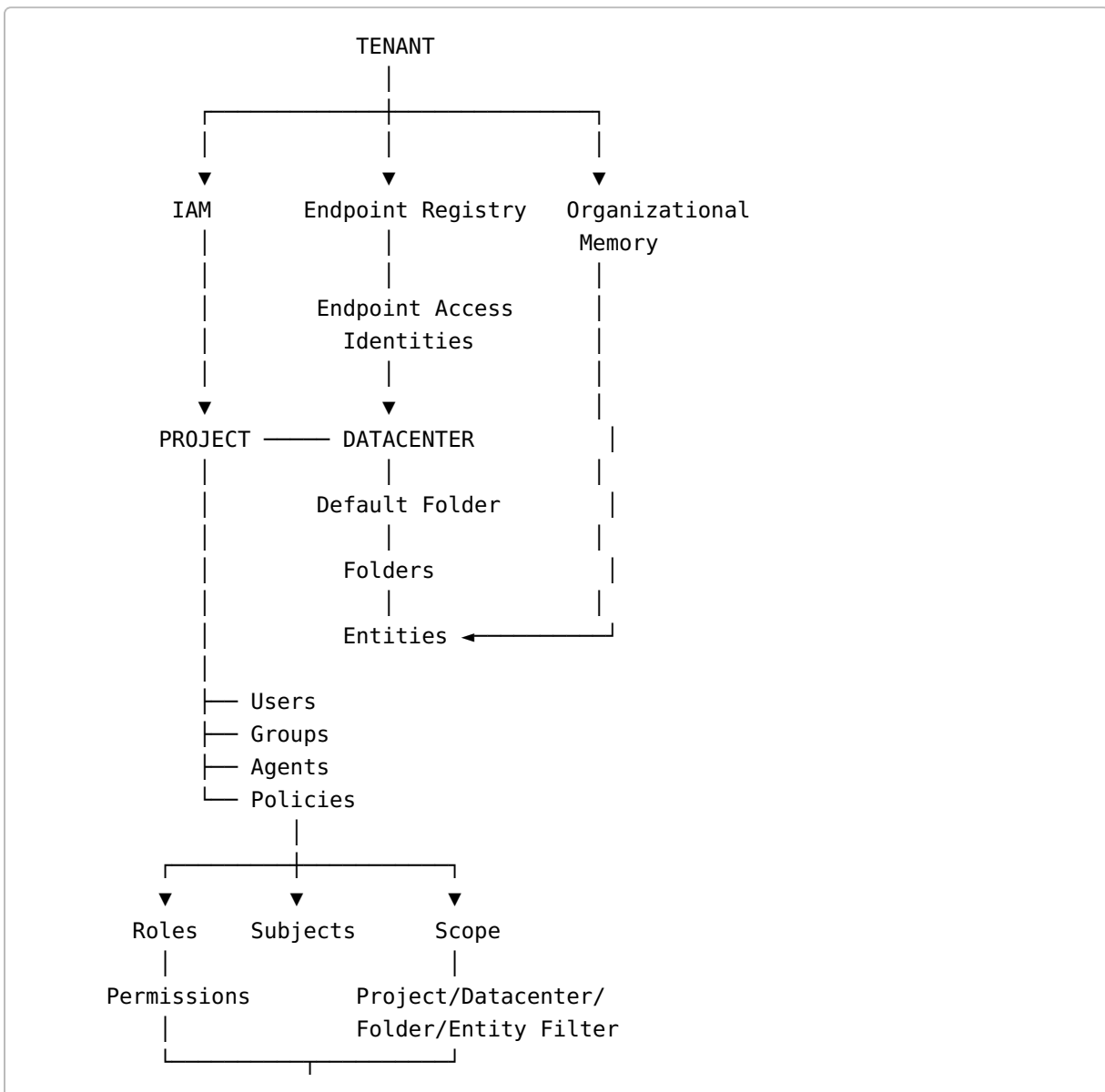
At every stage:

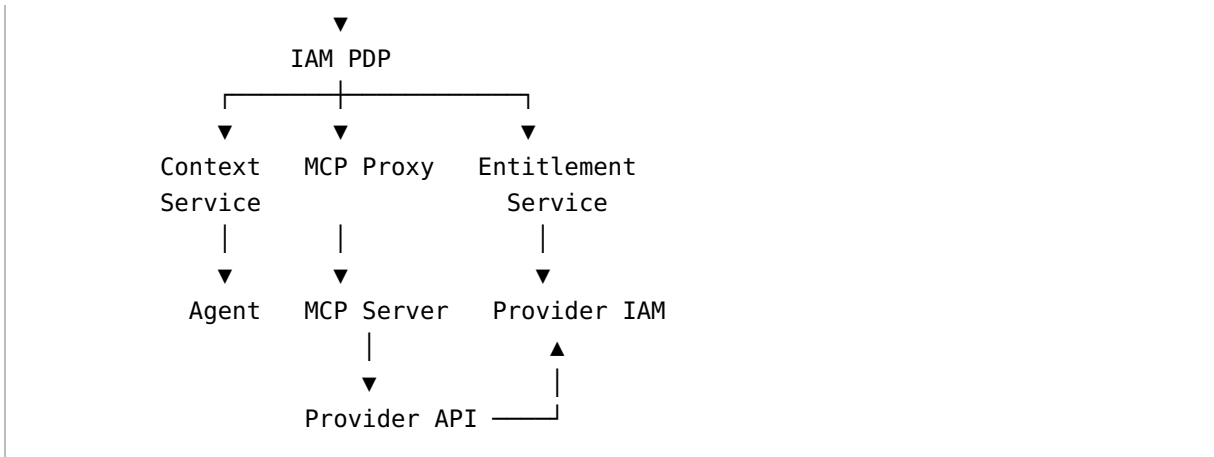


Datacenter
↓
Default Folder
↓
Folder
↓
Entity

remains unchanged.

98. Canonical Architecture





The physical implementation of these boxes may be consolidated for Desktop and decomposed for Hosted deployment.

99. Canonical Architectural Decisions

AD-1: Every Tenant has Project 0.

AD-2: A Project may contain multiple Datacenters.

AD-3: Every Datacenter has one mandatory Default Folder.

AD-4: Folders may branch at arbitrary depth.

AD-5: Datacenter is a first-class authorization and navigation boundary.

AD-6: Provider-native topology and AccuSec Folder hierarchy remain separate.

AD-7: Provider namespaces must not be ambiguously mixed.

AD-8: Endpoint and Datacenter are distinct objects.

AD-9: Endpoint Connections may be reused across Projects through Project-local Datacenters.

AD-10: One Endpoint may have multiple Endpoint Access Identities.

AD-11: Endpoint identities may have different provider privilege envelopes.

AD-12: Endpoint identity selection is deterministic and never controlled by an LLM.

AD-13: AccuSec administration does not imply provider administration.

AD-14: Provider administration does not imply AccuSec administration.

AD-15: Infrastructure Administrator and Virtual Project Administrator are separate roles.

AD-16: Users, groups and agents may receive access at Project, Datacenter and Folder level.

AD-17: Every AI Engineer belongs to exactly one Project.

AD-18: An AI Engineer may have different permissions across Datacenters within its Project.

AD-19: Agents require explicit human authorization.

AD-20: Agents cannot administer IAM/security.

AD-21: Roles contain permissions but Access Control Policies grant scoped authority.

AD-22: Access Control Policy consists of Role + Subjects + Scope.

AD-23: Scope Filters support structured logical AND.

AD-24: Policies support multiple line items and Boolean/set expressions.

AD-25: Complex Policy logic uses deterministic expression trees.

AD-26: NOT is bounded inside an authorized universe.

AD-27: MCP tools generate fine-grained executable permissions.

AD-28: MCP discovery never grants entitlement.

AD-29: Canonical operations remain distinct from MCP permissions.

AD-30: Multiple MCP Servers may exist per Project and Datacenter.

AD-31: MCP execution binds to a specific Datacenter, Endpoint and target Entity.

AD-32: Provider-operational Roles may require native entitlement provisioning.

AD-33: AccuSec Role and Provider Entitlement Profile are distinct.

AD-34: AccuSec stores desired entitlement state; the provider exposes actual enforcement state.

AD-35: Provider entitlements are reconciled for drift.

AD-36: Entitlement provisioning identities should be separated from execution identities.

AD-37: Entity sharing does not transfer ownership or source permissions.

AD-38: Every Entity retains one canonical identity.

AD-39: Every Tenant has one logical Organizational Memory.

AD-40: Organizational Memory never spans Tenants.

AD-41: Organizational Memory may include enterprise documents and files.

AD-42: Source ACLs are preserved.

AD-43: Context Service enforces IAM and ACL before content reaches an LLM.

AD-44: RAG/vector retrieval cannot bypass authorization.

AD-45: Security-relevant Memory labels and classifications are controlled and audited.

AD-46: Approval is separate from authorization.

AD-47: Provider IAM remains an independent native enforcement boundary.

AD-48: Desktop and Hosted editions use the same logical management and security architecture.

AD-49: Single-Tenant is cardinality one, not absence of the Tenant object.

AD-50: Single-user is cardinality one, not absence of Principal identity.

AD-51: Desktop supports multiple Projects, Datacenters, Folders, Endpoints, Endpoint identities and AI Engineers.

AD-52: Desktop may physically consolidate logical services but must preserve logical contracts.

AD-53: Hosted deployment may physically decompose logical services without changing domain semantics.

AD-54: Desktop-to-Hosted migration must not require redesigning IAM, tenancy, Endpoint, Entity or Organizational Memory models.

100. Core Security and Scale Invariants

1. Every deployment operates within at least one Tenant.
2. Desktop contains exactly one active Tenant.

3. Hosted multi-tenant may contain multiple Tenants.
4. Every Tenant has exactly one logical Organizational Memory.
5. Every Tenant has Project 0.
6. Every Project may contain multiple Datacenters.
7. Every Datacenter belongs to exactly one Project.
8. Every Datacenter has exactly one Default Folder.
9. Folders cannot cross Datacenter boundaries.
10. Provider identity must be retained during navigation and execution.
11. Authentication does not imply Project membership.
12. Project membership does not imply Datacenter permission.
13. Datacenter permission does not imply another Datacenter's permission.
14. AccuSec administration does not imply provider operational permission.
15. Provider operational permission does not imply AccuSec administration.
16. AI Engineers are independent Principals.
17. AI Engineers belong to exactly one Project.
18. AI Engineers cannot administer security.
19. Agents cannot switch to a more privileged Endpoint identity.
20. Delegated permissions cannot exceed grantor delegable permissions.
21. Roles alone do not grant access.
22. Policies bind Roles, Subjects and Scope.
23. Scope may exist at Project, Datacenter, Folder, Entity or Dynamic Entity Set level.
24. Scope Filters may resolve zero, one or many Entities.
25. Policy expressions are deterministic.

26. MCP tool discovery never grants entitlement.
27. Protected MCP execution requires explicit permission.
28. MCP execution resolves Datacenter and Endpoint.
29. Endpoint Connections may be reused across Projects where allowed.
30. Endpoint credentials are not exposed simply because a Principal can use the Endpoint.
31. One Endpoint may expose multiple access identities.
32. Endpoint identity selection is deterministic.
33. Provider entitlement provisioning cannot grant beyond authorized AccuSec intent.
34. Provider entitlement drift must be observable.
35. Provider authorization remains mandatory.
36. Entity sharing does not transfer ownership.
37. Shared Entities retain a single canonical Entity ID.
38. Destination authorization is independently evaluated.
39. Organizational Memory cannot be accessed outside its Tenant.
40. Source ACLs survive document ingestion.
41. Unauthorized Memory must not reach the model.
42. Agent ownership does not imply agent access to the owner's data.
43. Context retrieval preserves Project, Datacenter, Provider and Entity boundaries.
44. Material authorization and entitlement decisions are auditable.
45. Security failures fail closed.
46. LLM output never constitutes authorization.
47. Desktop retains explicit Tenant IDs.
48. Desktop retains explicit Principal IDs.

49. Desktop does not bypass IAM merely because there is one user.
 50. Desktop supports multiple Projects.
 51. Desktop supports multiple Datacenters per Project.
 52. Desktop supports multiple Folders per Datacenter.
 53. Desktop supports multiple Endpoint Connections.
 54. Desktop supports multiple Endpoint Access Identities.
 55. Desktop supports multiple MCP Servers.
 56. Desktop supports multiple AI Engineers.
 57. Hosted supports the same logical hierarchy with increased cardinality.
 58. Physical service consolidation does not eliminate logical component boundaries.
 59. Deployment mode does not alter fundamental authorization semantics.
 60. Desktop-to-Hosted migration preserves the core domain and security model.
-

101. Initial Functional Acceptance Criteria

Desktop Profile

The product can demonstrate:

- one local Tenant;
- one Human Principal;
- multiple Projects;
- multiple Datacenters;
- multiple provider-native hierarchy views;
- multiple Folder structures;
- multiple Endpoints;
- multiple Endpoint identities;
- multiple MCP Servers;
- multiple AI Engineers;
- one Tenant-local Organizational Memory;
- deterministic IAM enforcement.

Hosted Single-Tenant Profile

The product can additionally demonstrate:

- multiple Human Principals;
- Groups;
- SSO;
- delegated administration;
- multi-user audit;
- provider entitlement provisioning.

Hosted Multi-Tenant Profile

The product can additionally demonstrate:

- multiple isolated Tenants;
- one Organizational Memory per Tenant;
- separate Tenant IAM namespaces;
- separate Projects and Datacenters;
- strict cross-Tenant isolation.

Projects

The product can:

- create multiple Projects;
- assign users;
- assign groups;
- create agents;
- associate multiple Datacenters.

Datacenters

The product can:

- create multiple Datacenters per Project;
- associate Provider Type;
- associate Endpoint;
- create mandatory Default Folder;
- expose provider-native navigation.

Folders

The product can:

- create arbitrary hierarchical Folder trees;

- assign Policy at Datacenter or Folder level;
- preserve Datacenter boundaries.

Endpoints

The product can:

- register reusable Endpoints;
- share eligible Endpoint references across Projects;
- manage multiple Endpoint Access Identities;
- protect credential material.

IAM

The product can:

- normalize Human Principals;
- create Roles;
- create Permissions;
- create Access Control Policies;
- apply Project/Datacenter/Folder scopes;
- enforce default deny.

AI Engineers

The product can:

- create multiple agents;
- bind each agent to one Project;
- assign different permissions by Datacenter;
- enforce owner delegation;
- block security administration.

MCP

The product can:

- register multiple MCP Servers;
- discover tools;
- generate executable permissions;
- map them to canonical operations;
- IAM-check each protected RunTool;
- bind execution to Datacenter and Endpoint.

Endpoint Entitlements

The product can:

- map AccuSec Roles to Provider Entitlement Profiles;
- provision supported provider entitlements;
- record desired versus actual state;
- revoke provider access;
- detect drift.

Organizational Memory

The product can:

- maintain exactly one logical Memory per Tenant;
- ingest infrastructure knowledge;
- ingest approved enterprise documents;
- preserve source ACLs;
- index information by Project, Datacenter, Folder, Entity and security metadata.

Organizational Context

The product can:

- authorize Context requests;
- filter by Tenant;
- filter by Project and Datacenter;
- enforce source ACLs;
- construct only authorized Context Packages.

Audit

The product can reconstruct:

- initiating Human;
- executing Agent;
- AccuSec Role;
- Permission;
- Policy;
- Project;
- Datacenter;
- Endpoint;
- Endpoint Access Identity;
- MCP Tool;
- target Entity;
- provider result.

102. Product Success Criterion

For every protected operation, AccuSec must deterministically answer:

Who is acting?

Which Tenant owns the operation?

Which Project contains the Principal?

Which Datacenter is targeted?

Which provider Endpoint is involved?

Which Folder and Entity scope applies?

Which Access Control Policy matched?

Which Role and Permission authorize the operation?

Which MCP tool implements it?

Which Endpoint Access Identity will execute it?

Was that identity legitimately provisioned?

Does native provider IAM permit the operation?

What Organizational Memory may the Principal access?

Which ACLs and classifications restrict Context?

Does the action require approval?

Can the complete chain be audited afterward?

And at the product architecture level:

Can the exact same domain model operate with one Tenant and one user on a laptop and with many Tenants and many users in a hosted service without redesigning IAM, agents, Datacenters, Endpoints, memory or execution?

If the answer is yes, the Tenancy, Datacenter, IAM, Endpoint Entitlement and Organizational Memory architecture is functioning as intended.

103. Final Scaling Principle

The product shall be built according to the following invariant:

```
Desktop
Single Tenant
Single User
  |
  ▼
Hosted Single Tenant
Multiple Users
  |
  ▼
Hosted Multi-Tenant
Multiple Tenants
Multiple Users
```

while maintaining:

```
Tenant
  ↓
Project
  ↓
Datacenter
  ↓
Default Folder
  ↓
Folder
  ↓
Entity
```

and the same:

```
Principal Model
IAM Model
Access Control Policy Model
Endpoint Model
Endpoint Identity Model
MCP Permission Model
```

AI Engineer Model
Organizational Memory Model
Organizational Context Model
Audit Model
Execution Authorization Model

across the full deployment continuum.

Desktop is enterprise architecture at cardinality one where appropriate—not a separate architecture.